YOLOv8-Based Object Detection with Kalman Filter Optimization for Ball Tracking in Wheeled Soccer Robot

Fikri Rivandi[1*], Feri Candra[2]
[1,2]Dept. of Informatics Engineering, Faculty of Engineering, Universitas Riau, Pekanbaru, Indonesia
[1]fikri.rivandi2583@student.unri.ac.id, [2]feri@eng.unri.ac.id

*Corresponding author: fikri.rivandi2583@student.unri.ac.id

Abstract—Deep learning-based object detection, such as YOLO, significantly improves accuracy in KRSBI-Beroda robots but introduces heavy computational loads that lead to high latency and movement instability on limited hardware. This research proposes a tracking optimization system that integrates YOLOv8 with a Kalman Filter using a frame-skipping strategy. In this architecture, YOLOv8 serves as a periodic detector while the Kalman Filter acts as a recursive estimator to maintain tracking continuity during skip phases. Experimental results on an ASUS K401U laptop demonstrate a substantial increase in processing speed from 13.60 FPS to 38.77 FPS. Furthermore, the system achieved zero tracking loss during occlusion scenarios and reduced mission completion time by 62%. This integration provides a robust solution for real-time robotic vision in dynamic environments.

Keywords: KRSBI-Beroda, YOLOv8, Kalman Filter, Frame Skipping, Object Tracking

---

INTRODUCTION

The Engineering Robotic Club (ERC) of Universitas Riau (UNRI) has consistently participated in the Wheeled Indonesian Soccer Robot Contest (KRSBI Beroda). A critical challenge in this competition is the robot's ability to track the ball and goals autonomously under dynamic conditions. While deep learning methods like YOLO offer superior accuracy compared to traditional HSV filtering, they demand high computational resources.

On the current hardware platform ASUS K401U with an Intel Core i5 7200U and Nvidia 940MX, continuous YOLO inference creates a computational bottleneck, leading to reduced frame rates and delayed actuator responses. In addition, frequent occlusion conditions, where the ball is partially or completely obstructed, often result in immediate tracking failure. To address these issues, this study integrates a Kalman Filter to estimate the trajectory of the ball, enabling the robot to maintain positional awareness even during detection gaps or occlusion events, thereby producing smoother and more efficient movement.

METHODOLOGY

A. YOLOv8 Implementation

The system utilizes a YOLOv8 model trained on 1,125 images captured via the robot's omnidirectional camera. The detector outputs a bounding box in the format (x_c, y_c, w, h, C), which is then passed as the Measurement Vector (Z_k) to the Kalman Filter.

B. Kalman Filter Configuration

The state vector (X) is defined to encompass position, velocity, and object dimensions:

$$X = [x, y, v_x, v_y, w, h]^T$$

The transition matrix (F) is modeled based on constant velocity assumptions:

$$F = \begin{bmatrix} 1 & 0 & dt & 0 & 0 & 0 \\ 0 & 1 & 0 & dt & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

The observation matrix (H) maps the state space to the measurement space, focusing on position and dimensions since velocity is not directly measured.

C. Prediction and Frame Skipping Strategy

To optimize hardware usage, the system implements a two-tiered approach:

1. Prediction Phase: The Kalman Filter projects the next state based on the previous motion model.
2. Correction Phase: The YOLOv8 detector is invoked every several frames (S_L) to provide new measurements for the update step. During frames where $frame_count \bmod S_L \neq 0$, the system relies solely on the Kalman Filter prediction, thereby significantly reducing the frequency of neural network inference.

D. Grid-Based Motion Control

Predicted coordinates are translated into discrete motion commands via a grid localization algorithm. The field of view is partitioned into action zones (e.g., center-forward, left-correction, search-rotation). This allows the ROS processing unit to publish string commands (e.g., "maju", "putarKiri") to the Arduino actuator unit with minimal latency.

RESULT AND DISCUSSION

A. Computational Efficiency (Static Test)

Static evaluation compared the processing speed of the baseline (continuous YOLO) versus the optimized (YOLO + KF) system over 10 repetitions.

System	Average FPS	Highest FPS
Baseline	13.60	15.09
Optimized	38.77	44.00

Table 1. Summary of FPS Performance

The results indicate that skipping heavy inference frames allows the system to reach much higher frame rates, effectively resolving the hardware bottleneck on the ASUS K401U platform.

B. Occlusion Robustness

The system was subjected to three occlusion scenarios: light (30%), moderate (60%), and momentary (1-2s total coverage).

Scenario	Baseline Loss Count	Optimized Loss Count
Moderate Occlusion	96	0
Momentary Occlusion	6	0

Duration of Tracking Loss	13.94 s	0s

Table 2. Tracking Continuity Metrics

While the baseline system suffered immediate "blindness" upon losing visual features, the optimized system maintained an internal estimation of the ball's position using velocity vectors (v_x, v_y). This enabled instantaneous re-tracking as soon as the object reappeared.

C. Dynamic Performance and Stability

In dynamic game scenarios, the optimized system demonstrated superior stability by reducing "jitter"—the rapid switching of commands due to noisy visual data.

Metric	Baseline System	Optimized System
Mission Time (Average)	15.17 seconds	5.77 seconds
Command Jitter (Turns)	74 counts	23 counts
Goal Rate	100%	100%

Table 3. Dynamic Mission and Command Log

The reduction in jitter indicates that the Kalman Filter's smoothing effect provides a linear and predictable input for the grid-based control logic. Consequently, the robot completed missions 62% faster by avoiding the erratic corrections seen in the baseline system.

CONCLUSION

This research proves that integrating YOLOv8 with a Kalman Filter provides a holistic performance boost for soccer robots with limited hardware. The proposed strategy achieves three major objectives: (1) drastically increasing frame rates via frame skipping, (2) ensuring tracking continuity through occlusion handling, and (3) enhancing movement stability via data smoothing. These improvements directly translate to faster, more competitive robotic performance in the dynamic KRSBI Beroda environment.

ACKNOWLEDGMENT

I would like to express my heartfelt gratitude to everyone who supported the completion of this article. Although I am unable to mention each individual by name, their contributions, encouragement, and assistance are sincerely appreciated.

REFERENCES

BIOGRAPHIES OF AUTHORS